

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

LAPORAN JOBSHEET 11

2.1 Praktikum 1: Pembuatan Single Linked List

→ Kode Program

> Kode program class Mahasiswa06.java

```
package Pertemuan11;
```

```
public class Mahasiswa06 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    Mahasiswa06() {  
  
    }  
  
    public Mahasiswa06(String nm, String name, String kls, double ip) {  
        this.nim = nm;  
        this.nama = name;  
        this.kelas = kls;  
        this.ipk = ip;  
    }  
  
    public void tampilInformasi() {  
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);  
    }  
}
```

> Kode program class Node06.java

```
package Pertemuan11;
```

```
public class Node06 {  
    Mahasiswa06 data;  
    Node06 next;  
  
    public Node06(Mahasiswa06 data, Node06 next) {  
        this.data = data;  
        this.next = next;  
    }  
}
```

> Kode program class SingleLinkedList06.java

```
package Pertemuan11;
```

```

public class SingleLinkedList06 {
    Node06 head;
    Node06 tail;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node06 tmp = head;
            System.out.println("Isi Linked List:\t");
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked List kosong");
        }
    }

    public void addFirst(Mahasiswa06 input) {
        Node06 ndInput = new Node06(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    public void addLast(Mahasiswa06 input) {
        Node06 ndInput = new Node06(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }

    public void insertAfter(String key, Mahasiswa06 input) {
        Node06 ndInput = new Node06(input, null);
        Node06 temp = head;
        do {
            if (temp.data.nama.equalsIgnoreCase(key)) {

```

```

        ndInput.next = temp.next;
        temp.next = ndInput;
        if (ndInput.next == null) {
            tail = ndInput;
        }
        break;
    }
    temp = temp.next;
} while (temp != null);
}

public void insertAt(int index, Mahasiswa06 input) {
    if (index < 0) {
        System.out.println("indeks salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node06 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node06(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}
}

```

> Kode program class SLLMain06.java

```

package Pertemuan11;

public class SLLMain06 {
    public static void main(String[] args) {
        SingleLinkedList06 sll = new SingleLinkedList06();
        Mahasiswa06 mhs1 = new Mahasiswa06("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa06 mhs2 = new Mahasiswa06("23212201", "Bimon", "2B", 3.8);
        Mahasiswa06 mhs3 = new Mahasiswa06("22212202", "Cintia", "3C", 3.5);
        Mahasiswa06 mhs4 = new Mahasiswa06("21212203", "Dirga", "4D", 3.6);

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
    }
}

```

→ Hasil Running

```
Linked List kosong
Isi Linked List:
Dirga  21212203      4D      3.6

Isi Linked List:
Dirga  21212203      4D      3.6
Alvaro 24212200      1A      4.0

Isi Linked List:
Dirga  21212203      4D      3.6
Cintia 22212202      3C      3.5
Bimon  23212201      2B      3.8
Alvaro 24212200      1A      4.0
```

Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?

Jawab: Karena pada saat program pertama kali dijalankan, linked list belum memiliki data atau node apapun. Nilai variabel head masih bernilai null, yang menandakan bahwa linked list dalam kondisi kosong. Di dalam method print(), terdapat pengecekan menggunakan kondisi !isEmpty(). Method isEmpty() akan mengembalikan nilai true apabila head == null. Karena kondisi tersebut terpenuhi, maka !isEmpty() bernilai false sehingga program tidak masuk ke blok if, melainkan masuk ke blok else dan menampilkan pesan “Linked List Kosong”. Hal ini terjadi karena belum ada proses penambahan data seperti addFirst() atau addLast() yang dipanggil sebelum method print() dijalankan.

2. Jelaskan kegunaan variable temp secara umum pada setiap method!

Jawab: Variabel temp digunakan sebagai penunjuk sementara (temporary pointer) dalam proses pengolahan linked list. Variabel ini berfungsi untuk menelusuri setiap node mulai dari head hingga node terakhir tanpa mengubah posisi head yang sebenarnya. Dengan menggunakan temp, program dapat melakukan berbagai operasi seperti menampilkan data, mencari node tertentu, serta membantu proses penambahan dan penghapusan node. Selama proses tersebut, temp akan berpindah dari satu node ke node berikutnya melalui referensi next hingga mencapai null.

3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

Jawab:

→ Kode Program

> Modifikasi kode program class SLLMain06.java

```
package Pertemuan11;
import java.util.Scanner;
```

```

public class SLLMain06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList06 sll = new SingleLinkedList06();

        System.out.print("Masukkan jumlah data: ");
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.println("Data mahasiswa ke-" + (i + 1));
            System.out.print("NIM\t: ");
            String nim = sc.nextLine();

            System.out.print("Nama\t: ");
            String nama = sc.nextLine();

            System.out.print("Kelas\t: ");
            String kelas = sc.nextLine();

            System.out.print("IPK\t: ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa06 mhs = new Mahasiswa06(nim, nama, kelas, ipk);
            sll.addLast(mhs);
        }
        System.out.println("\nData Linked List: ");
        sll.print();
    }
}

```

→ Hasil Running

```

Masukkan jumlah data: 4
Data mahasiswa ke-1
NIM   : 24212200
Nama  : Alvaro
Kelas : 1A
IPK   : 4,0
Data mahasiswa ke-2
NIM   : 23212201
Nama  : Bimon
Kelas : 2B
IPK   : 3,8
Data mahasiswa ke-3
NIM   : 22212202
Nama  : Cintia
Kelas : 3C
IPK   : 3,5
Data mahasiswa ke-4

```

NIM : 21212203
Nama : Dirga
Kelas : 4D
IPK : 3,6

Data Linked List:

Isi Linked List:

Alvaro	24212200	1A	4.0
Bimon	23212201	2B	3.8
Cintia	22212202	3C	3.5
Dirga	21212203	4D	3.6

2.2 Praktikum 2: Modifikasi Elemen pada Single Linked List

→ Kode Program

> Kode program Mahasiswa06.java

```
package Pertemuan11;
```

```
public class Mahasiswa06 {
```

```
    String nim;  
    String nama;  
    String kelas;  
    double ipk;
```

```
    Mahasiswa06() {
```

```
}
```

```
    public Mahasiswa06(String nm, String name, String kls, double ip) {
```

```
        this.nim = nm;  
        this.nama = name;  
        this.kelas = kls;  
        this.ipk = ip;  
    }
```

```
    public void tampilInformasi() {
```

```
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);  
    }
```

```
}
```

> Kode program Node06.java

```
package Pertemuan11;
```

```
public class Node06 {
```

```
    Mahasiswa06 data;  
    Node06 next;
```

```
    public Node06(Mahasiswa06 data, Node06 next) {
```

```

        this.data = data;
        this.next = next;
    }
}

```

> Kode program SingleLinkedList06.java

```

package Pertemuan11;

public class SingleLinkedList06 {
    Node06 head;
    Node06 tail;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node06 tmp = head;
            System.out.println("Isi Linked List:\t");
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked List kosong");
        }
    }

    public void addFirst(Mahasiswa06 input) {
        Node06 ndInput = new Node06(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    public void addLast(Mahasiswa06 input) {
        Node06 ndInput = new Node06(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }
}

```

```
}
```

```
public void insertAfter(String key, Mahasiswa06 input) {  
    Node06 ndInput = new Node06(input, null);  
    Node06 temp = head;  
    do {  
        if (temp.data.nama.equalsIgnoreCase(key)) {  
            ndInput.next = temp.next;  
            temp.next = ndInput;  
            if (ndInput.next == null) {  
                tail = ndInput;  
            }  
            break;  
        }  
        temp = temp.next;  
    } while (temp != null);  
}
```

```
public void insertAt(int index, Mahasiswa06 input) {  
    if (index < 0) {  
        System.out.println("indeks salah");  
    } else if (index == 0) {  
        addFirst(input);  
    } else {  
        Node06 temp = head;  
        for (int i = 0; i < index - 1; i++) {  
            temp = temp.next;  
        }  
        temp.next = new Node06(input, temp.next);  
        if (temp.next.next == null) {  
            tail = temp.next;  
        }  
    }  
}
```

```
public void getData(int index) {  
    Node06 tmp = head;  
    for (int i = 0; i < index; i++) {  
        tmp = tmp.next;  
    }  
    tmp.data.tampilInformasi();  
}
```

```
public int indexOf(String key) {  
    Node06 tmp = head;  
    int index = 0;  
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {  
        tmp = tmp.next;  
        index++;  
    }  
}
```



```

        if (tmp == null) {
            return -1;
        } else {
            return index;
        }
    }

    public void removeFirst() {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
        } else if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
        }
    }

    public void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
        } else if (head == tail) {
            head = tail = null;
        } else {
            Node06 temp = head;
            while (temp.next != tail) {
                temp = temp.next;
            }
            temp.next = null;
            tail = temp;
        }
    }

    public void remove(String key) {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat dihapus!");
        } else {
            Node06 temp = head;
            while (temp != null) {
                if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head)) {
                    this.removeFirst();
                    break;
                } else if (temp.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }

```

```

    }
}

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        Node06 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
}
}
}

```

> Kode program SLLMain06.java

```

package Pertemuan11;
import java.util.Scanner;

public class SLLMain06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList06 sll = new SingleLinkedList06();

        System.out.print("Masukkan jumlah data: ");
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.println("Data mahasiswa ke-" + (i + 1));
            System.out.print("NIM\t: ");
            String nim = sc.nextLine();

            System.out.print("Nama\t: ");
            String nama = sc.nextLine();

            System.out.print("Kelas\t: ");
            String kelas = sc.nextLine();

            System.out.print("IPK\t: ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa06 mhs = new Mahasiswa06(nim, nama, kelas, ipk);
            sll.addLast(mhs);
        }
    }
}

```

```

        System.out.println("\nData Linked List: ");
        sll.print();

        System.out.println("data index 1 : ");
        sll.getData(1);

        System.out.println("data mahasiswa an Bimon berada pada index : " +
        sll.indexOf("bimon"));
        System.out.println();

        sll.removeFirst();
        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();
    }
}

```

→ Hasil Running

```

data index 1 :
Cintia 22212202      3C      3.5
data mahasiswa an Bimon berada pada index : 2

Isi Linked List:
Cintia 22212202      3C      3.5
Bimon  23212201      2B      3.8

Isi Linked List:
Bimon  23212201      2B      3.8

```

Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!

Jawab: Pada fungsi remove, keyword break digunakan karena berfungsi untuk menghentikan perulangan setelah node yang dicari berhasil ditemukan dan dihapus dari linked list. Setelah node tersebut dihapus, tidak diperlukan lagi pengecekan ke node berikutnya karena tujuan sudah tercapai. Selain itu, penggunaan break membuat proses menjadi lebih efisien karena menghindari perulangan yang tidak diperlukan.

2. Jelaskan kegunaan kode dibawah pada method remove

```

1  temp.next = temp.next.next;
2  if (temp.next == null) {
3      tail = temp;
4  }

```

Jawab: Baris `temp.next = temp.next.next;` digunakan untuk menghapus sebuah node dari linked list dengan cara melewati node tersebut. Pointer next dari node sebelumnya (`temp`) diarahkan langsung ke node setelah node yang akan dihapus, sehingga node tersebut tidak lagi terhubung dalam linked list. Selanjutnya, kondisi `if (temp.next == null)` digunakan untuk mengecek apakah node yang dihapus merupakan node terakhir. Jika setelah penghapusan nilai `temp.next` menjadi null, maka node `temp` menjadi node terakhir. Oleh karena itu, variabel `tail` diperbarui dengan `tail = temp` agar tetap menunjuk ke node terakhir yang benar.

Tugas

→ Kode Program

> Kode program class `DataMahasiswa06.java`

```
package Pertemuan11;
```

```
public class DataMahasiswa06 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public DataMahasiswa06(String nm, String name, String kls, double ip) {
        this.nim = nm;
        this.nama = name;
        this.kelas = kls;
        this.ipk = ip;
    }

    public void tampilInformasi() {
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

> Kode program class `NodeMahasiswa06.java`

```
package Pertemuan11;
```

```
public class NodeMahasiswa06 {
    DataMahasiswa06 data;
    NodeMahasiswa06 next;

    public NodeMahasiswa06(DataMahasiswa06 data, NodeMahasiswa06 next) {
        this.data = data;
        this.next = next;
    }
}
```

> Kode program class `QueueLinkedList06.java`

```
package Pertemuan11;
```

```

public class QueueLinkedList06 {
    NodeMahasiswa06 front, rear;
    int size = 0;

    boolean isEmpty() {
        return front == null;
    }

    boolean isFull() {
        return false;
    }

    void enqueue(DataMahasiswa06 data) {
        NodeMahasiswa06 newNode = new NodeMahasiswa06(data, null);
        if (isEmpty()) {
            front = rear = newNode;
        } else {
            rear.next = newNode;
            rear = newNode;
        }
        size++;
        System.out.println("Mahasiswa masuk antrian");
    }

    void dequeue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong");
        } else {
            System.out.println("Mahasiswa dipanggil.");
            front.data.tampilInformasi();
            front = front.next;
            size--;
        }
    }

    void peekFront() {
        if (!isEmpty()) {
            front.data.tampilInformasi();
        } else {
            System.out.println("Antrian kosong");
        }
    }

    void peekRear() {
        if (!isEmpty()) {
            rear.data.tampilInformasi();
        } else {
            System.out.println("Antrian kosong");
        }
    }
}

```

```

void print() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        NodeMahasiswa06 temp = front;
        while (temp != null) {
            temp.data.tampilInformasi();
            temp = temp.next;
        }
    }
}

int getSize() {
    return size;
}

void clear() {
    front = rear = null;
    size = 0;
    System.out.println("Antrian dikosongkan");
}
}

```

> Kode program class QueueMain06.java

```

package Pertemuan11;
import java.util.Scanner;

public class QueueMain06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLinkedList06 q = new QueueLinkedList06();

        int pilih;
        do {
            System.out.println("\n=== MENU ANTRIAN MAHASISWA ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Panggil Antrian");
            System.out.println("3. Tampilkan Semua Antrian");
            System.out.println("4. Lihat Antrian (Terdepan & Terakhir)");
            System.out.println("5. Jumlah Antrian");
            System.out.println("6. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilih = sc.nextInt();
            sc.nextLine();

```

```

switch (pilih) {
    case 1:
        System.out.print("NIM  : ");
        String nim = sc.nextLine();
        System.out.print("Nama  : ");
        String nama = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();
        System.out.print("IPK   : ");
        double ipk = sc.nextDouble();
        sc.nextLine();

        DataMahasiswa06 mhs = new DataMahasiswa06(nim, nama, kelas, ipk);
        q.enqueue(mhs);
        break;

    case 2:
        q.dequeue();
        break;

    case 3:
        q.print();
        break;

    case 4:
        System.out.println("Antrian Terdepan:");
        q.peekFront();
        System.out.println("Antrian Terakhir:");
        q.peekRear();
        break;

    case 5:
        System.out.println("Jumlah antrian: " + q.getSize());
        break;

    case 6:
        q.clear();
        break;

    case 0:
        System.out.println("Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid");
}

```

```

        break;
    }

    } while (pilih != 0);
}
}

```

→ Hasil Running

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 1

NIM : 24212200

Nama : Alvaro

Kelas : 1A

IPK : 4,0

Mahasiswa masuk antrian

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 1

NIM : 23212201

Nama : Bimon

Kelas : 2B

IPK : 3,8

Mahasiswa masuk antrian

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 1

NIM : 22212202

Nama : Cintia
Kelas : 3C
IPK : 3,5
Mahasiswa masuk antrian

==== MENU ANTRIAN MAHASISWA ====

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 1

NIM : 21212203

Nama : Dirga

Kelas : 4D

IPK : 3,6

Mahasiswa masuk antrian

==== MENU ANTRIAN MAHASISWA ====

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 2

Mahasiswa dipanggil:

Alvaro 24212200 1A 4.0

==== MENU ANTRIAN MAHASISWA ====

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 3

Bimon 23212201 2B 3.8

Cintia 22212202 3C 3.5

Dirga 21212203 4D 3.6

==== MENU ANTRIAN MAHASISWA ====

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian

4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 4

Antrian Terdepan:

Bimon 23212201 2B 3.8

Antrian Terakhir:

Dirga 21212203 4D 3.6

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 5

Jumlah antrian: 3

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 6

Antrian dikosongkan

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 3

Antrian kosong!

=== MENU ANTRIAN MAHASISWA ===

1. Tambah Antrian
2. Panggil Antrian
3. Tampilkan Semua Antrian
4. Lihat Antrian (Terdepan & Terakhir)
5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 0

Terima kasih!